

KIT DE CONSTRUCCIÓN · EJECUTAR AL GANAR EL CAPITAL SEMILLA (O ANTES, PARA LA DEMO)

VÍA Salud — Manual de prompts

16 prompts listos para copiar y pegar en **Claude Code**, en orden, para construir la plataforma completa: del repositorio vacío al servicio operando con clientes. Cada prompt es una sesión de trabajo con su criterio de "listo". El plan calza con lo declarado a Sercotec: MVP en 90 días, versión 2 en los meses 4-6.

Fase 0 · El contexto maestro (CLAUDE.md) — 1 prompt

Fase 1 · MVP, meses 1-3 · esqueleto, datos, WhatsApp, asistente IA, rescate de cupos, panel, demo, seguridad, deploy — 10 prompts

Fase 2 · V2, meses 4-6 · predicción de no-show, llamadas de voz, reporte mensual — 3 prompts

Fase 3 · Escala, meses 7+ · onboarding express, calculadora pública — 2 prompts

Herramienta: Claude Code (claude.ai/code o la app de escritorio) sobre un repositorio git propio y privado. Stack elegido: Python + FastAPI + PostgreSQL — un solo lenguaje cubre la API, el asistente y el modelo de predicción. Agosto 2026.

Cómo usar este manual (leer antes del primer prompt)

- **Un prompt por sesión, en orden.** Cada uno asume que los anteriores están hechos. No saltarse el Prompt 0: es el que le da contexto a todos los demás.
- **Al terminar cada prompt:** pedir a Claude que corra los tests, probarlo tú mismo siguiendo el criterio de "listo", y hacer commit en git antes de pasar al siguiente. Si algo no funciona, se lo describes en la misma sesión ("al hacer X pasa Y") y lo corrige.
- **Puedes pedir explicaciones siempre:** "explícame qué hace este archivo como si yo no fuera programador" es un prompt válido en cualquier momento.
- **Nunca uses datos reales de pacientes durante el desarrollo.** Todo se construye y prueba con los datos ficticios del modo demo (Prompt 8). Los datos reales entran solo en producción, con un centro firmado y su consentimiento.
- **Las claves (API keys, contraseñas) nunca van en el código** ni en los prompts: van en variables de entorno. Claude lo hará así si sigues estos prompts tal cual.
- **Adelanto útil:** los Prompts 0, 1, 2, 5-parcial y 8 bastan para tener la **demo del video y del Comité**. Se pueden ejecutar hoy mismo, antes de los resultados.

FASE 0 — El contexto maestro

PROMPT 0 · Crear el repositorio y el CLAUDE.md del proyecto

Cuándo: Hoy mismo. Primera sesión, repositorio vacío. · **Listo cuando:** Existe CLAUDE.md en la raíz y Claude lo resume correctamente al preguntarle qué es el proyecto.

PROMPT — COPIAR Y PEGAR COMPLETO

Crea un repositorio para VÍA Salud y escribe un archivo CLAUDE.md en la raíz que sirva de contexto para todas las sesiones futuras, con este contenido (redáctalo bien, no lo copies literal):

QUÉ ES: VÍA Salud es una plataforma SaaS chilena que reduce las listas de espera de salud recuperando las horas médicas que se pierden por inasistencia. Clientes: centros médicos y dentales privados medianos (5-30 boxes) del Gran Concepción y, más adelante, salud municipal (CESFAM). El sistema: (1) importa la agenda del centro a un calendario espejo, (2) confirma cada cita por WhatsApp con un asistente de IA (y luego por llamada automática), (3) detecta citas en riesgo y, cuando una hora se libera, la ofrece en minutos a una "lista de rescate" priorizada, (4) registra todo en una bitácora auditable y lo muestra en un panel con horas y pesos recuperados.

REGLAS INQUEBRANTABLES DEL PRODUCTO: datos mínimos de pacientes (nombre, teléfono, cita, motivo general — NUNCA ficha clínica ni diagnósticos detallados); toda acción queda en la bitácora append-only; ninguna decisión clínica es automática (la priorización la configura y valida el centro); el asistente jamás da consejos médicos — ante cualquier consulta clínica deriva al centro; cumplimiento Ley 19.628 y Ley 21.719 (el centro es responsable del tratamiento, VÍA Salud es encargado); multi-centro desde el día 1 (cada centro ve solo lo suyo).

STACK: Python 3.12 + FastAPI + PostgreSQL + SQLAlchemy/Alembic; frontend del panel con plantillas Jinja + HTMX (simple, sin framework pesado); tests con pytest; docker-compose para desarrollo; deploy en Railway. Asistente conversacional con el SDK oficial de Anthropic.

MODELO DE NEGOCIO (para que entiendas prioridades): suscripción mensual \$150.000-\$350.000 por centro; el valor está en el rescate de cupos y en el reporte de horas recuperadas. El norte de todo el producto es un número: horas recuperadas al mes por centro.

Crea también la estructura básica del repo (src/, tests/, docker-compose.yml con Postgres, pyproject.toml, .env.example, .gitignore correcto) y un README breve. Aún sin lógica de negocio.

FASE 1 — MVP (meses 1-3): lo declarado a Sercotec

PROMPT 1 · Modelo de datos y migraciones

Cuándo: Tras el Prompt 0. · **Listo cuando:** docker-compose levanta Postgres, las migraciones corren, y los tests de modelos pasan.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Diseña e implementa el modelo de datos con SQLAlchemy + migraciones Alembic:

- centro (id, nombre, comuna, zona horaria, configuración de mensajes y de prioridad como JSON, plan contratado)
- usuario_centro (login del administrador/recepción, rol admin|repcion, hash de contraseña)
- profesional (centro, nombre, especialidad, box)
- paciente_contacto (centro, nombre, teléfono normalizado E.164, consentimiento_registrado bool + fecha; SIN datos clínicos)
- cita (centro, profesional, paciente_contacto, fecha_hora, duración, estado: agendada|confirmada|en_riesgo|liberada|rescatada|atendida|perdida, motivo_general opcional, origen_import)
- lista_rescate (centro, paciente_contacto, especialidad deseada, ventana horaria preferida, prioridad numérica, origen: adelanto|sin_cupo|manual, activo)
- oferta_rescate (cita liberada, paciente_contacto, enviada_at, expira_at, estado: enviada|aceptada|rechazada|expirada)
- conversacion y mensaje (por paciente y canal, con dirección in/out, cuerpo, estado de entrega, needs_human bool)
- evento_bitacora: APPEND-ONLY (id, centro, timestamp, tipo, entidad, entidad_id, payload JSON, hash_anterior, hash — cadena de hashes SHA-256 para que sea evidente cualquier alteración). Sin UPDATE ni DELETE: hazlo cumplir con un trigger de Postgres.

Índices para las consultas frecuentes (citas por centro+fecha, ofertas activas). Tests de modelos y del trigger de inmutabilidad. Todo multi-centro: cada query de negocio filtra por centro_id.

PROMPT 2 · Calendario espejo: importador de agenda CSV/Excel

Cuándo: Tras el Prompt 1. · **Listo cuando:** Subo un Excel de agenda de ejemplo y aparecen las citas del día; re-importar no duplica.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Implementa el calendario espejo: los centros exportan su agenda diaria desde su software (Dentalink, AgendaPro, Reservo o planillas propias) a CSV/Excel, y VÍA Salud la importa.

- Endpoint y pantalla simple para subir el archivo (por centro), más un comando CLI equivalente.
- Parser tolerante: detecta columnas por nombre flexible (fecha, hora, paciente, teléfono, profesional, especialidad) con un mapeo configurable por centro que se guarda la primera vez.
- Normaliza teléfonos chilenos a E.164 (+56...); marca los inválidos como "incontactables" (no los descarta: son insumo de depuración para el centro).
- Idempotente: re-importar el mismo día actualiza en vez de duplicar (clave natural centro+profesional+fecha_hora+paciente). Detecta citas que desaparecieron del archivo (canceladas en origen) y las marca liberadas.

- Cada import queda en la bitácora (cuántas filas, nuevas, actualizadas, liberadas, inválidas).
 - Genera 2 archivos de ejemplo con datos FICTICIOS (uno estilo planilla dental, otro estilo centro médico) y tests que los importan.
- No hay integración por API con terceros en esta etapa: el CSV es el contrato.

PROMPT 3 · Integración WhatsApp (API oficial, modo sandbox primero)

Cuándo: Tras el Prompt 2. Requiere cuenta en un partner de la API de WhatsApp Business (Twilio u otro BSP autorizado por Meta). · **Listo cuando:** En sandbox: recibo en mi teléfono la plantilla de confirmación y mi respuesta queda registrada en la conversación.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Implementa la capa de WhatsApp sobre la API oficial de WhatsApp Business vía Twilio (deja la interfaz abstraída en un módulo canal_whatsapp para poder cambiar de BSP después):

- Envío de plantillas aprobadas (confirmación inicial) y de mensajes de sesión (respuestas dentro de la ventana de 24 h).
- Webhook de entrada: recibe mensajes y estados de entrega (enviado/entregado/leído/fallido), valida la firma del webhook, persiste en conversacion/mensaje y actualiza la bitácora.
- Cola de envío con reintentos y rate limiting (usa la base de datos como cola simple; sin Redis por ahora).
- Modo SANDBOX por defecto (número de prueba de Twilio) controlado por variable de entorno; en desarrollo, además un "canal falso" que imprime los mensajes en consola para tests sin red.
- Plantilla inicial de confirmación (texto propuesto): "Hola {nombre}, le recordamos su hora en {centro} el {fecha} a las {hora} con {profesional}. Responda 1 para CONFIRMAR, 2 si NO PUEDE ASISTIR, o escríbanos su duda."
- Credenciales solo por variables de entorno; nada en el código. Tests con el canal falso.

PROMPT 4 · El asistente conversacional con Claude (confirmar, cancelar, reagendar)

Cuándo: Tras el Prompt 3. · **Listo cuando:** Converso por WhatsApp sandbox: '1' confirma; 'no puedo ese día' libera la hora y ofrece reagendar; una pregunta rara escala a humano.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Implementa el cerebro conversacional con el SDK oficial de Anthropic (paquete `anthropic`):

- Respuestas simples ("1", "2", "sí", "no puedo") se resuelven con reglas, sin llamar a la API: rápido y gratis.
- Todo lo demás va a Claude con tool use estricto (strict: true). Modelo por defecto: claude-opus-5; deja el modelo configurable por variable de entorno y documenta en el README que las conversaciones rutinarias pueden enrutar a claude-haiku-4-5 por costo cuando haya volumen. Maneja el stop_reason "refusal" con un mensaje neutro que deriva al centro.
- Herramientas del asistente: confirmar_cita, marcar_no_asiste(motivo_general), proponer_reagendo(opciones que le pasas tú desde la agenda), aceptar_reagendo, actualizar_telefono, escalar_a_humano(razon), responder_info_practica(direccion|preparacion|horario — con textos configurados por el centro).
- System prompt del asistente (redáctalo con esto): eres el asistente de {centro}; tono cercano, chileno, breve, trato de usted; SOLO gestionas citas e información práctica; JAMÁS des consejo médico, diagnóstico ni interpretación de síntomas — ante cualquier consulta clínica o urgencia responde que un humano del centro le contactará y usa escalar_a_humano; no pidas ni registres datos de salud; si el paciente se molesta o pide humano, escala de inmediato.
- Escalamiento temporal de confirmación: mensaje a las 72 h antes, recordatorio a las 24 h si no responde, último aviso 4 h antes; si nunca responde, marcar cita en_riesgo. Programador de tareas (APScheduler) que dispara estos envíos.

- Toda acción del asistente queda en la bitácora. Tests de los flujos con la API simulada (mock) y un test de integración opcional con la API real marcado como lento.

PROMPT 5 · El motor de rescate de cupos (el corazón del producto)

Cuándo: Tras el Prompt 4. · **Listo cuando:** Canelo una cita de prueba y en minutos el siguiente de la lista recibe la oferta; si no responde en el plazo, pasa al siguiente; el panel muestra la hora como rescatada.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Implementa el motor de rescate — el diferenciador del producto:

- Disparadores de liberación: paciente cancela por WhatsApp, import marca cita desaparecida, o recepción libera manualmente desde el panel.
- Al liberarse una hora: construir la cola de candidatos desde lista_rescate filtrando por especialidad compatible y ventana horaria, ordenada por prioridad (configurable por el centro: por defecto orden de inscripción + preferencia declarada; SIN criterios clínicos automáticos).
- Oferta secuencial: WhatsApp al candidato 1 ("se liberó una hora el {fecha} {hora} con {profesional}, ¿la quiere? Responda SÍ para tomarla") con expiración configurable (por defecto 20 minutos); si expira o rechaza, pasa al siguiente; si acepta, crea la cita, actualiza la agenda espejo, saca al paciente de la lista de rescate y NOTIFICA al personal del centro (WhatsApp o correo) para que lo registre en su software.
- Alimentación de la lista de rescate: (a) recepción la carga desde el panel, (b) el asistente ofrece "¿quiere que le avisemos si se libera una hora antes?" a quien reagenda para más adelante, (c) pacientes que pidieron hora sin cupo.
- Métricas por centro que esto debe dejar calculables: horas liberadas, ofertas enviadas, tasa de aceptación, tiempo mediano de rescate, horas rescatadas y su valor en pesos (duración × tarifa configurada del centro).
- Todo en la bitácora. Tests del flujo completo con el canal falso, incluyendo expiración y cola vacía.

PROMPT 6 · Panel web del centro

Cuándo: Tras el Prompt 5. · **Listo cuando:** Entro con usuario y contraseña y veo: horas recuperadas del mes, \$ recuperados, inasistencia por profesional, cola de rescate y conversaciones que piden humano.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Construye el panel web (Jinja + HTMX, diseño sobrio y legible, en español):

- Login por centro con roles admin/recepción; sesiones seguras; cada centro ve exclusivamente lo suyo.
- PORTADA (lo que vende): tarjetas grandes del mes en curso — horas rescatadas, pesos recuperados (horas × tarifa del centro), tasa de inasistencia del mes vs. mes anterior, citas confirmadas hoy. Gráfico simple de inasistencia por semana.
- Agenda del día con estado de cada cita (confirmada / en riesgo / liberada / rescatada) y acción manual de liberar/reasignar.
- Cola de rescate: ver, agregar y priorizar pacientes manualmente.
- Bandeja "necesita humano": conversaciones escaladas por el asistente, con historial completo y campo para responder (sale por el mismo canal WhatsApp).
- Configuración del centro: textos de info práctica, tarifa promedio por hora (para valorizar), horarios de envío, tiempos de expiración de ofertas, mapeo de columnas del import.
- Vista de bitácora con filtros y exportación CSV (el argumento de auditabilidad para el mundo municipal).
- Pantalla de importación de agenda (la del Prompt 2) integrada. Tests de permisos: un centro nunca ve datos de otro.

PROMPT 7 · Reporte mensual y correo de resultados

Cuándo: Tras el Prompt 6. · **Listo cuando:** Se genera un PDF de un centro de prueba con sus horas y pesos recuperados del mes.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Implementa el reporte mensual por centro — el documento que sostiene la renovación y la reunión mensual:

- PDF de 2-3 páginas generado con weasyprint: resumen del mes (horas rescatadas, pesos recuperados, inasistencia antes/después, ofertas y tasa de aceptación), tabla por profesional, y la lista de pacientes incontactables detectados (teléfonos malos) como valor agregado de depuración.
- Comparación con los 3 meses anteriores cuando exista historial.
- Generación automática el día 1 de cada mes (tarea programada) + botón "generar ahora" en el panel + envío por correo al administrador (SMTP configurable).
- Números idénticos a los del panel: misma fuente de cálculo, un solo módulo de métricas compartido.

PROMPT 8 · Modo demo (para el video, el Comité y las ventas)

Cuándo: Puede adelantarse: solo requiere Prompts 0-5. · **Listo cuando:** En un teléfono cualquiera puedo mostrar en 2 minutos: confirmación, cancelación y rescate de un cupo, con datos ficticios.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Crea el modo demo:

- Comando `make demo` que puebla un centro ficticio ("Centro Médico Demo", profesionales y pacientes inventados con teléfonos de prueba) y deja el sistema en sandbox.
- Guion de demo automatizado: un script que, paso a paso y con pausas, (1) envía la confirmación a MI número real de prueba, (2) al responder "no puedo", libera la hora, (3) me envía la oferta de rescate a un segundo número (o al mismo), (4) al responder "sí", muestra en el panel la hora rescatada y la bitácora del ciclo completo.
- Un README-DEMO.md de una página con el paso a paso para ejecutarla frente a alguien (Comité de Sercotec, un administrador de centro) sin improvisar.
- Botón "reiniciar demo" que limpia y repuebla los datos ficticios.

PROMPT 9 · Seguridad y cumplimiento (Ley 19.628 / 21.719)

Cuándo: Tras el Prompt 6, antes del primer cliente real. · **Listo cuando:** La revisión de seguridad de Claude no encuentra problemas críticos y existe el checklist de cumplimiento completado.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Haz una pasada completa de seguridad y cumplimiento:

- Revisa TODO el código como auditor: inyección SQL, autenticación y sesiones, autorización multi-centro (fuga entre centros), validación del webhook, secretos, dependencias. Corrige lo que encuentres y lista lo corregido.
- Cifrado: TLS en todo, cifrado en reposo del teléfono del paciente (columna cifrada), contraseñas con hash fuerte.
- Minimización y ciclo de vida: job de borrado/anonimización de conversaciones y contactos según retención configurable (por defecto 12 meses); exportación de datos de un centro si se va (portabilidad).
- Registro de consentimiento: campo y flujo para que el centro declare que informó a sus pacientes; primera línea del primer mensaje incluye identificación del centro y opción de no recibir más mensajes (opt-out que se respeta siempre).

- Escribe docs/cumplimiento.md: checklist Ley 19.628 y 21.719 con el rol de cada parte (centro = responsable, VÍA Salud = encargado), y un borrador de anexo de tratamiento de datos para el contrato con cada centro (para revisión de abogado, dilo explícito).
- Backups automáticos diarios de la base con restauración probada por un test.

PROMPT 10 · Deploy a producción

Cuándo: Tras el Prompt 9. Requiere cuenta en Railway (o similar) y un dominio. · **Listo cuando:** La app corre en <https://app.viasalud.cl> (o el dominio elegido), con backups y alertas; la demo funciona apuntando a producción en sandbox.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Deja el sistema en producción:

- Deploy en Railway: servicio web + Postgres gestionado + worker de tareas programadas; variables de entorno documentadas en .env.example; migraciones automáticas al desplegar.
- Dominio con HTTPS, healthcheck, y página de estado simple.
- Observabilidad mínima que un fundador solo pueda mirar: logs estructurados, alerta por correo/WhatsApp si el webhook falla, si la cola de envío se atasca o si un backup no corre.
- Entornos separados: staging (sandbox de WhatsApp) y producción; el modo demo disponible en ambos.
- docs/runbook.md: qué hacer si se cae, cómo restaurar backup, cómo rotar claves, cómo dar de alta un centro nuevo paso a paso.

FASE 2 — V2 (meses 4-6): lo declarado como versión 2

PROMPT 11 · Predicción de no-show

Cuándo: Cuando existan 4-8 semanas de datos reales de los pilotos. · **Listo cuando:** Cada cita del día muestra su nivel de riesgo y el escalamiento de mensajes se adapta a él.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Implementa la predicción de inasistencia en dos etapas:

- ETAPA A (hoy, sin datos): score heurístico configurable por reglas — puntos por: historial de inasistencias del paciente, antigüedad de la cita (más días desde que se agendó = más riesgo), día/hora (lunes primera hora, viernes tarde), no respondió la confirmación de 72 h, primera visita vs. control. Documenta los pesos y déjalos ajustables por centro.
- ETAPA B (con datos de pilotos): entrena un modelo gradient boosting (scikit-learn) con las citas históricas y sus resultados; features = las mismas de la heurística + las que aporten los datos; validación temporal (entrenar en meses pasados, evaluar en el siguiente); guarda el modelo por centro o global según volumen; deja un comando de reentrenamiento mensual y registra métricas (AUC, precisión en el decil más riesgoso) en la bitácora.
- Uso del score en ambos casos: intensificar confirmación de citas riesgosas (mensaje extra, llamada en V2 de voz, aviso a recepción) y priorizar qué horas vigilar para el rescate. El score se muestra en el panel como bajo/medio/alto con sus razones (explicable). NUNCA se usa para negar ni quitar una hora a nadie: solo para confirmar mejor.

PROMPT 12 · Llamadas de voz automatizadas (adultos mayores)

Cuándo: Tras el Prompt 11. · **Listo cuando:** Un número de prueba recibe la llamada: voz clara lee la cita y confirmo marcando 1 o diciendo 'sí'.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Agrega el canal de voz con Twilio Voice (misma abstracción de canal que WhatsApp):

- Flujo de confirmación por llamada: texto-a-voz en español chileno claro y pausado ("Le llamamos de {centro} para confirmar su hora del {fecha} a las {hora}. Presione 1 para confirmar, 2 si no puede asistir, o 3 para repetir"), captura por teclado (DTMF) y, si el proveedor lo soporta bien, también por voz simple (sí/no).
- Cuándo llamar (configurable): pacientes sin WhatsApp, sin respuesta al chat en 24 h, o citas de alto riesgo según el score del Prompt 11. Horarios de llamada respetuosos (10:00-19:00) y máximo de intentos.
- Resultado de la llamada (confirmó / canceló / no contestó / buzón) actualiza la cita, dispara rescate si cancela, y queda en bitácora y en el panel.
- Costos por minuto registrados por centro para el control de margen. Tests con el proveedor simulado.

PROMPT 13 · Sobrecupo selectivo (opcional, con el centro de acuerdo)

Cuándo: Cuando la predicción (Prompt 11, etapa B) esté validada. · **Listo cuando:** El panel sugiere bloques con sobrecupo recomendado y el centro puede aceptarlo o ignorarlo.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Implementa el sobrecupo selectivo como SUGERENCIA, nunca automático:

- Con el modelo de no-show, identificar bloques donde la probabilidad de al menos una ausencia es alta (ej. > 80%) y sugerir en el panel "este bloque admite 1 sobrecupo con bajo riesgo", con la explicación del cálculo.
- El centro decide: si acepta, la cita extra se agenda marcada como sobrecupo; el sistema monitorea y, si nadie falta, ofrece proactivamente reagendo compensado al sobrecupo (mensaje de disculpa + prioridad máxima en la lista de rescate).
- Métricas de honestidad: cuántos sobrecupos sugeridos, aceptados, y cuántas veces "falló" (todos llegaron). Si la tasa de fallo supera el umbral configurado, el sistema deja de sugerir en ese centro y lo dice.

FASE 3 — Escala (meses 7+)

PROMPT 14 · Onboarding de un centro nuevo en menos de 1 día

Cuándo: Cuando haya 3+ clientes y el alta manual empiece a doler. · **Listo cuando:** Doy de alta un centro completo (datos, mapeo, plantillas, usuarios) en menos de una hora de trabajo real.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Convierte el alta de un centro en un flujo guiado:

- Asistente de onboarding en el panel (solo para mí, rol superadmin): crear centro → cargar primer CSV y mapear columnas con vista previa → configurar tarifa, textos y horarios → crear usuarios → enviar mensaje de prueba → checklist de salida con todo verificado.
- Plantillas por tipo de centro (dental / médico) que precargan configuraciones típicas.
- Página interna de salud por centro: última importación, tasa de entrega de WhatsApp, ofertas activas, errores recientes — para detectar problemas antes que el cliente.

PROMPT 15 · Calculadora pública de horas perdidas (marketing)

Cuándo: En paralelo a la venta, cuando el sitio web del proyecto esté listo. · **Listo cuando:** Una página pública donde un administrador ingresa 3 números y ve cuánta plata pierde al mes, con botón de contacto.

PROMPT — COPIAR Y PEGAR COMPLETO

Lee CLAUDE.md. Crea una página pública estática (puede vivir en el mismo dominio) con la calculadora de horas perdidas:

- Inputs: citas al mes, % de inasistencia estimado (con el 15,6% nacional precargado como referencia), valor promedio de la consulta.
- Output inmediato y visual: horas perdidas al mes, pesos perdidos al mes y al año, y cuánto de eso recupera VÍA Salud con el rango conservador (40-60%).
- Botón "quiero mi prueba de 30 días" que abre WhatsApp conmigo con un mensaje prellenado que incluye los números que ingresó.
- Sin registro, sin cookies de más, carga instantánea, se ve bien en el teléfono (el administrador la abrirá desde el celular en la misma reunión).

Reglas de oro mientras construyes

Regla	Por qué
Terminar cada prompt con tests verdes y un commit	Si algo se rompe después, siempre hay un punto seguro al que volver
Probar tú mismo el criterio de "listo" antes de avanzar	Los tests no reemplazan usar el producto como lo usará el cliente
Datos ficticios hasta tener contrato y consentimiento	Cumplimiento 19.628/21.719 y cero riesgo reputacional
Claves solo en variables de entorno	Un repo con claves filtradas es una crisis evitable
No agregar funciones fuera de este plan hasta terminar la Fase 1	El MVP declarado a Sercotec en 90 días es el compromiso; lo demás espera
Pedir a Claude "explícame qué hiciste y por qué" al final de cada sesión	Tú tienes que poder defender el sistema ante el Comité y ante clientes

El orden mínimo para la demo de esta semana: Prompt 0 → 1 → 2 → 3 → 4 → 8 (el rescate del Prompt 5 puede simularse en la demo si aún no está). Con eso grabas el video del asistente real y tienes qué mostrar en la visita a terreno y ante el Comité.